CTU

CZECH TECHNICAL UNIVERSITY IN PRAGUE

Faculty of Electrical Engineering

Department of Cybernetics

Bachelor thesis

Distance estimate of autonomous helicopters by analysis of signal strength

Fedor Strjapunin

Prague, May 2026

Supervisor: Ing. Martin Zoula

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Strjapunin** Jméno: **Fedor** Osobní číslo: **516180**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Odhad vzdálenosti autonomních helikoptér pomocí analýzy kvality signálu

Název bakalářské práce anglicky:

Distance estimate of autonomous helicopters by analysis of signal strength

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Martin Zoula Multirobotické systémy FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **27.01.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

Jan Kybic
Digitálně podepsal(a) Jan Kybic
Datum: 05.02.2026 13:51:21
podpis vedoucí(ho) ústavu/katedry

Veronika Sobotíková
Digitálně podepsal(a) Veronika Sobotíková
Datum: 05.02.2026 23:53:48
ID: 31854
podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání: 06.02.2026

Strjapunin Fedor
Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Strjapunin** Jméno: **Fedor** Osobní číslo: **516180**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Odhad vzdálenosti autonomních helikoptér pomocí analýzy kvality signálu

Název bakalářské práce anglicky:

Distance estimate of autonomous helicopters by analysis of signal strength

Pokyny pro vypracování:

The goal is to analyse experimental data about location and signal strength [1,4] from the flight of autonomous helicopters with the focus on reproducible research principles [3].
1. Get familiar with existing tools [2] and methods [3] of reproducible research.
2. Discuss existing tools for reproducible research support and, if necessary, propose and implement a new system.
3. Get acquainted with real flight data of two autonomous helicopters of the Multi-robot Systems group..
4. Evaluate the relationship between wireless signal strength and relative location of the two aircraft. Employ appropriate statistical evaluation while following the reproducible research principles

Seznam doporučené literatury:

[1]. Liu T, Zhao H, Yang H, Zheng K, Chatzimisios P. “Design and implementation of a novel real-time unmanned aerial vehicle localization scheme based on received signal strength.” Trans Emerging Tel Tech. 2021; 32:e4350. doi: 10.1002/ett.4350
[2]. Agrawal A, Scolnick M. “marimo - an open-source reactive notebook for Python.” Available from "https://github.com/marimo-team/marimo".
[3]. The Turing Way Community, “The Turing Way: A handbook for reproducible, ethical and collaborative research”. Zenodo, Apr. 14, 2025. doi: 10.5281/zenodo.15213042.
[4]. R. S. Yokoyama, B. Y. L. Kimura, and E. dos Santos Moreira, “An architecture for secure positioning in a UAV swarm using RSSI-based distance estimation,” SIGAPP Appl. Comput. Rev., vol. 14, no. 2, pp. 36–44, June 2014, doi: 10.1145/2656864.2656867. Available: http://dx.doi.org/10.1145/2656864.2656867

FAKULTA ELEKTROTECHNICKÁ
FACULTY OF ELECTRICAL ENGINEERING
Technická 2
166 27 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Strjapunin Fedor
Personal number:	516180
Programme name:	Cybernetics and Robotics

hereby declare that the Bachelor's Thesis titled

Distance estimate of autonomous helicopters by analysis of signal strength

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 21.05.2026

Fedor Strjapunin
.....
student's signature

Acknowledgement

I would like to thank my supervisor, Martin Zoula, for his guidance, valuable feedback and revisions, provided without delay, which was invaluable in the moments of extreme time pressure. I would also like to thank my fiancée for her unwavering support, encouragement and understanding, especially during the last and most demanding stages of the writing process.

Abstract

This thesis aims to improve RSSI-based distance estimation between autonomous UAVs by compensating for the joint radiation pattern of the transmitting and receiving UAVs, which could potentially provide acceptable ranging accuracy while using ubiquitous hardware. An approach that uses a k-nearest neighbors regressor to model the joint radiation pattern and incorporates it into the distance calculation is proposed and implemented. The approach is evaluated on real flight data of autonomous UAVs provided by the Multi-Robot Systems group at the Czech Technical University in Prague. Within individual flights, an average RMSE of approximately 7.6 meters was achieved, reducing the error by approximately 47 percent compared to the baseline (14.3 meters). However, these results must be interpreted with caution because cross-flight validation was not possible due to significant hardware differences between flights. As a secondary contribution, existing tools for reproducible research were reviewed, and a computational graph creation tool was proposed and used to implement all data-processing steps prioritizing their reproducibility.

Keywords

RSSI, distance estimation, UAV, antenna radiation pattern, signal strength, Kalman filter, k-nearest neighbors, reproducible research

Abstrakt

Tato práce si klade za cíl zlepšit přesnost odhadu vzdálenosti mezi autonomními UAV na základě RSSI pomocí kompenzace jejich společného vyzařovacího diagramu, což by mohlo potenciálně poskytnout přijatelnou přesnost měření využitím běžně přítomného hardwaru. Je navržen postup, který využívá KNN regresor k modelování společného vyzařovacího diagramu a využívá této znalosti při výpočtu vzdálenosti. Postup je vyhodnocen na reálných letových datech autonomních UAV poskytnutých skupinou Multirobotických systémů na Českém vysokém učení technickém v Praze. V rámci jednotlivých letů bylo dosaženo průměrné RMSE přibližně 7,6 metru, což představuje snížení chyby o přibližně 47 procent oproti základnímu modelu (14,3 metru). Tyto výsledky je však třeba interpretovat opatrně, protože křížová validace napříč lety nebyla možná z důvodu významných hardwarových rozdílů mezi jednotlivými lety. Jako sekundární přínos byly přezkoumány existující nástroje pro reprodukovatelný výzkum a byl navržen a použit nástroj pro vytváření výpočetních grafů, který je využit k implementaci všech kroků týkajících se zpracování dat s důrazem na jejich reprodukovatelnost.

Klíčová slova

RSSI, odhad vzdálenosti, UAV, vyzařovací diagram antény, kvalita signálu, Kalmanův filtr, k nejbližších sousedů, reprodukovatelný výzkum

Contents

List of Figures

List of Tables

Glossary

A2A Air-to-Air.
API Application Programming Interface.
ArUco Augmented Reality University of Cordoba.

BDDS Bearing Distance Diversity Score.
BLE Bluetooth Low Energy.

CEX Caching Aware Executor.

DAG Directed Acyclic Graph.
DS-TWR Double-Sided Two-Way Ranging.

GNSS Global Navigation Satellite System.

IEEE Institute of Electrical and Electronics Engineers.

JRP Joint Radiation Pattern.
JRPC Joint Radiation Pattern Compensation.

LiDAR Light Detection and Ranging.
LoS Line of Sight.

MAE Mean Absolute Error.
MRS Multi-Robot Systems.

P95E 95th Percentile Error.

RMSE Root-Mean-Square Error.
RPC Remote Procedure Call.
RSSI Received Signal Strength Indicator.

ToF Time of Flight.
TWR Two Way Ranging.

UAV Unmanned Aerial Vehicle.
UI User Interface.
UWB Ultra-wide Band.

VSC Visual Studio Code.

w.r.t with respect to.

Symbols

G_{rx} Receiver antenna gain (dBi).
G_{tx} Transmitter antenna gain (dBi).
P_{rx} Received signal power (dBm).
P_{tx} Transmitted signal power (dBm).
$\hat{P}_{\text{rx}}$ Received signal power (W).
$\hat{P}_{\text{tx}}$ Transmitted signal power (W).
λ Wavelength (m).

CHAPTER 1

Introduction

Estimating the distance between autonomous Unmanned Aerial Vehicles (UAVs) is essential in many aspects of controlling their flight. These include maintaining flight formation [1], proximity detection, collision avoidance, where the minimum separation between UAVs is upheld based on mutual ranging, and defense against location spoofing by attackers, where the reported location of a node is verified by a verifier node based on distance measurements provided by other nodes [2].

In areas where absolute Global Navigation Satellite System (GNSS) positioning is denied, local ranging methods are employed to obtain the relative distance. Examples include Ultra-wide Band (UWB) Two Way Ranging (TWR) [3], Light Detection and Ranging (LiDAR), ranging based on Augmented Reality University of Cordoba (ArUco) marker detection [4] and are briefly discussed in Chapter 2. This thesis focuses on Received Signal Strength Indicator (RSSI) based estimation, which uses ubiquitous hardware and could potentially provide ranging at no extra increase to the cost or payload of the UAV. Additionally, it is not significantly affected by weather conditions, such as rainfall [5] and humidity [6].

In RSSI based ranging, the power of the received 2.4 GHz WiFi signal is measured and then converted to distance using the known transmitted power and predicted path loss for the environment. In free space, the power of the received signal is inversely proportional to the square of the distance from the source, as described by the Friis transmission equation [7]:

$$\hat{P}_{\text{rx}} = \hat{P}_{\text{tx}}\,\hat{G}_{\text{rx}}\,\hat{G}_{\text{tx}}\left(\frac{\lambda}{4\pi d}\right)^2, \tag{1.1}$$

where $\hat{P}_{\text{rx}}$ is the received power in watts, $\hat{P}_{\text{tx}}$ the transmitted power in watts, $\hat{G}_{\text{tx}}$ the transmitter antenna gain (linear, dimensionless) in the direction of the receiver, $\hat{G}_{\text{rx}}$ the receiver antenna gain (linear, dimensionless) in the direction of the transmitter, λ the wavelength in meters, and d the distance between antennas in meters.

However, the direction-dependent antenna gains, seen in the Friis equation 1.1, can vary based on the azimuth-elevation angles of the transmitter UAV with respect to (w.r.t) the heading of the receiver UAV and azimuth-elevation angles of the receiver UAV w.r.t the heading of the transmitter UAV. For the sake of brevity, these four angles will be referred to as mutual bearing in this thesis. As the mutual bearing changes, the gains are affected. This is caused by the onboard antennas not being truly isotropic, meaning that they neither transmit nor receive signals with uniform power in all directions, as well as potential obstruction of the Line of Sight (LoS) between antennas by the body of either UAV. The combination of the direction-dependent gains $\hat{G}_{\text{tx}}$ and $\hat{G}_{\text{rx}}$ will be referred to as the joint radiation pattern. Other potential sources of interference such as multipath propagation [8] discussed in Section 2.2 are neglected.

The aim of this thesis is to create a model that, based on the mutual bearing, predicts the joint radiation pattern. The predicted radiation pattern is then incorporated into the RSSI based distance calculation to improve distance estimate accuracy. Real-world UAV flight data, provided by the Multi-Robot Systems (MRS) group, is used for the development and evaluation of the joint radiation pattern prediction model.

The provided data contains multiple flights, each recording a pair of UAVs. A separate joint radiation pattern prediction model was trained and evaluated for each pair of UAVs. The evaluation was performed by comparing the average Root-Mean-Square Error (RMSE), Mean Absolute Error (MAE) and 95th Percentile Error (P95E) of distance estimates (with joint radiation pattern compensation) against a baseline that estimates distance directly from filtered RSSI readings. In some flights, because the distance between UAVs is constant or does not vary for similar mutual bearings, it is possible to obtain an accurate distance estimate from the mutual bearing alone, without using RSSI. Therefore, an additional evaluation against a control model (that directly predicts distance from bearing) was also conducted.

The results showed that compensating for the joint radiation pattern improves performance over the baseline, reducing the average RMSE by 6.67 meters (from 14.27 meters to 7.60 meters). On flights where mutual bearing alone is not sufficient to accurately predict distance, the proposed model achieved substantially lower errors than both the baseline and the control model. This indicates that the joint radiation pattern prediction model is indeed learning a useful, distance-invariant mapping from mutual bearing to the effective gain. Cross-flight validation was also attempted, however, due to hardware differences (e.g., varying antenna alignment), generalization across multiple flights proved impossible.

1.1 Software and reproducible research

An additional aim of this work is to make the development and evaluation of the joint radiation pattern prediction model reproducible as defined by the Turing Way Handbook [9]. The Handbook defines reproducible research as work where, given the original code and data, it is possible to independently recreate the same results. While it does not prescribe universal principles, it offers concrete guidelines; this thesis adopts several of them to ensure transparency and repeatability.

First, the Handbook stresses that raw data should be preserved unchanged and made available. Accordingly, the processing pipelines developed in this thesis only ever read and derive from the original data but never modify it. Because the full flight recordings provided by the MRS group exceed 50 GB, only the relevant flight data, such as GNSS coordinates, RSSI readings, and others, are provided instead of the complete recordings.

Second, the Handbook recommends version control for all analysis code. All data-processing scripts and pipeline definitions are therefore kept under version control, ensuring that the computational steps can be exactly repeated.

Third, the Handbook advises automating the generation of results to eliminate manual copying errors. In this work, all tables of quantitative results are produced automatically by the pipeline, removing any risk of hand-entry mistakes.

Finally, based on the discussion of existing tools used to support reproducible research in Section 2.5, a new tool for the creation of computational graphs is proposed, implemented, and used for the development of the joint radiation pattern prediction model.

CHAPTER 2

Related works

In this section, an overview of selected related works is provided. To illustrate the commonly employed ranging methods, a brief overview is given. Compensating for direction-dependent antenna gains is discussed as well as Received Signal Strength Indicator (RSSI) filtering. An overview of existing software tools for reproducible research support is also given.

2.1 Local ranging methods

An example of a local ranging method is Time of Flight (ToF) based Ultra-wide Band (UWB) Two Way Ranging (TWR) [3]. UWB calculates ToF by measuring the round-trip time of a packet sent via radio, typically on either of two frequency bands: 3.2 GHz to 4.7 GHz or 6.2 GHz to 8.7 GHz. The ranging is performed as follows:

1. The initiator sends a poll message, and records a timestamp T_1 of when the poll message was sent.
2. The counterpart then receives it, notes the time of reception as T_2, and after possibly waiting for a set delay period of T_3, sends a message that includes T_2 and T_3 to the initiator.
3. The initiator, after receiving a response, notes the time of reception as T_4
4. The total ToF is then calculated as $\frac{(T_4-T_1)-(T_3-T_2)}{2}$

With an unobstructed line-of-sight, UWB can provide distance estimates with the precision of decimeters. The two-way ranging method relies on precise frequency synchronization between the initiator and responder clocks. If one clock runs slightly faster, an error is introduced into the ToF measurement, and the accuracy of the distance estimate can degrade. This can be further mitigated by using a more robust method of Double-Sided Two-Way Ranging (DS-TWR) [10].

Another method is Light Detection and Ranging (LiDAR), which obtains laser ToF based on the delay between emitting a pulse and detecting its reflection via a photodetector. While measuring via LiDAR provides high-frequency, accurate measurements and does not require cooperation like UWB, it has a disadvantage. Determining which object caused the reflection is non-trivial and requires a lot of computationally intensive data post-processing, which is scarce in mobile robotics. This can lead to LiDAR based ranging systems mistaking a bird, branch, or another object for the Unmanned Aerial Vehicle (UAV). In general, this makes LiDAR more suitable for environment geometry mapping/collision avoidance rather than mutual localization.

Marker-based methods are also employed, where a camera detects a marker (ArUco [4] or other), computer vision techniques are employed to read its ID, encoded as a black-and-white two-dimensional pattern, and subsequently uses known marker size and pixel coordinates of

its corners to compute rotation and translation of the marker relative to the camera. This provides identification, distance, and bearing information in one measurement. In general, marker or other vision-based methods are dependent on environmental factors, such as lighting and weather conditions, which can significantly degrade performance. They are also limited by the camera's resolution, which causes the performance to degrade when the range increases.

2.2 Multipath and ground reflection effects

An effect called multipath propagation occurs when a transmitted signal reflects from physical objects, resulting in the receiver not only receiving the transmitted signal but also several reflected copies of it [8]. These copies can cause constructive or destructive interference. As a result, the RSSI reading may temporarily increase or decrease.

For this work, since it focuses on ranging between two UAVs flying at high altitude, the most relevant source of multipath propagation might be Air-to-Air (A2A) signal ground reflection. Two relevant articles explore the effects of floor type and altitude on ground reflections. The first deals with grass, soil, and rubber floor [11]. The second explores the forest and seawater floor [12].

The effects of ground reflections were investigated by conducting an experiment, where the transmitter UAV is stationary at a fixed height, and the receiver UAV is flying in an "elevator" pattern, varying only its height. The path loss values (difference between transmitted and received power) were then compared to two models, free-space path loss log distance model (used in this thesis) and A2A two-ray model, which takes ground reflections into account. The results have shown that, for all surfaces, at sufficient altitude (the highest was approximately 10 meters for water, as it seems to reflect radio signals strongly) the ground reflection effect is diminished such that the path loss follows the free-space model. This supports our hypothesis that at sufficient altitudes the main obstacle for obtaining an accurate distance estimate is the effect of the combined radiation pattern of receiver and transmitter, combined with obstruction of Line of Sight (LoS) between antennas by the bodies of the UAVs.

2.3 RSSI filtering

A paper [13] by Emanuele Pagliari et al. about real-time UAV localization proposed an algorithm for passive UAV position estimation, where RSSI readings from multiple ground access points are used to calculate the position of a UAV by employing a multilateration algorithm. While the multilateration method is not directly relevant to this thesis since we only work with one UAV to estimate the distance of another, the paper does state that by combining Kalman filters with the estimation algorithm, it was possible to achieve a modest reduction in mean prediction error and a qualitative improvement in the results. According to the authors, the Kalman filters removed outlier measurements and smoothed the RSSI values without discarding relevant information.

Four setups in total were adopted in [13]: multilateration from raw RSSI, multilateration from pre-filtered RSSI, multilateration from raw RSSI with filtering of the final estimate, and multilateration with both prefiltering RSSI and filtering the final estimate. The table 2.1 shows the mean and 95th percentile prediction errors for a mobile UAV, along with the employed filtering setup. The reduction of mean prediction error from 5.49 meters to 5.37 meters, 95th percentile error from 9.99 meters to 9.35 meters, and the reported qualitative

improvement indicate that we should employ RSSI prefiltering.

Filtering Method	**Mean Position Error** (m)	**95-th Percentile Error** (m)
RAW RSSI	5.49	9.99
RAW RSSI with 2nd KF	5.23	9.06
RSSI with 1st KF	5.37	9.35
RSSI with both KFs	5.34	8.52

Table 2.1: Mean position error and 95th percentile position error of a multilateration algorithm for a mobile UAV with different filtering setups [13].

Another paper [14] by Rafhanah Shazwani Rosli et al. compared the performance of three filters used for RSSI readings. Simple moving average filter (SMA), alpha trimmed mean filter (ATM), and Kalman filter were tested on three criteria: noise reduction, data proximity, and delays using the ESP8266 WiFi module. The RSSI readings were recorded by setting up the receiver and transmitter with a direct LoS at a distance of 1 meter, and then recording with both clear and momentarily obstructed LoS by crossing it. The conditions are relevant to our work, since the LoS might be obstructed by the UAV's body. The most relevant criterion for our use case, noise reduction, was measured by computing the standard deviation of the RSSI reading samples. The Kalman filter performed best, showing the lowest standard deviation before, after, and during obstruction.

2.4 Direction-dependent antenna gain compensation

Some connectivity solutions, like Bluetooth Low Energy (BLE), include native RSSI based ranging [15], but its accuracy suffers from antenna orientation effects [16]. Patent [17] acknowledges that antenna orientation degrades the accuracy of RSSI-based distance estimates. To refine distance estimation between a mobile device and objects such as cellular base stations, WiFi access points, and FM broadcast stations, it proposes obtaining the device's orientation via onboard sensors and using a pre-calibrated antenna gain pattern lookup table to compensate for orientation-induced error in the RSSI readings. The patent does not provide any quantitative data, but it does claim to improve location accuracy by compensating for antenna orientation. This suggests that our proposed approach was already attempted with success in simpler conditions.

2.5 Existing software for reproducible research support

Before attempting an implementation of a custom data tool for creating computational graphs or data processing pipelines, it is important to analyze the gaps in the existing solutions. Existing popular tools for reproducible research can be roughly categorized as: computational notebooks, build systems, workflow orchestrators, and Python task decorators. In the following subsections, a brief discussion is provided for each category of tools along with relevant examples.

2.5.1 Computational notebooks

Computational notebooks are custom editors that allow users to create code cells and interactive UI elements and execute them either one at a time or the entire notebook at once. They require only a small amount of learning and setup and provide useful features such as immediate display of artifacts. In general, they are well-suited for exploration and prototyping. Python specific examples include Jupyter[18] and Marimo[19]. Notably, Jupyter uses IPython kernel [20] to execute the Python code, which allows it to persist variable contents between cell executions and capture the execution artifacts.

The disadvantage is that it is difficult to organize code well, because it is essentially written in a single file. The dependencies between cells are also unclear, and it is not immediately apparent which downstream cells consume data generated by the parent cell. In the case of Jupyter, running cells out of order may yield different results, which leads to reproducibility issues [21].

Marimo [19] attempts to solve this problem by invalidating all dependent cells whenever a variable that is consumed by them is assigned/reassigned by the user. It cannot, however, detect mutations, such as reassigning an object property or a column in a numpy matrix. It also provides another interesting feature by storing the notebooks as valid Python scripts.

2.5.2 Build systems

Build systems like Make [22] function by modeling dependencies between files and using file modification times to determine which parts of a project are out of date. When a prerequisite is newer than its target, make executes the user-defined recipe to rebuild only what changed, thereby avoiding unnecessary recompilation and providing efficient incremental execution. Another example is CMake (Cross-platform Make). It is a platform-agnostic meta build system that takes the user-defined build rules and transpiles them to a number of target platforms such as Unix Makefiles, Ninja, Visual Studio, Xcode, and others, which can potentially save effort if building on multiple platforms is required.

While build systems are widely used in software development projects, several downsides exist when using them for computational experiments. First, they require users to learn a new syntax. Second, they require the user to maintain a meta-program alongside their actual experiments. Third, as build rules grow in complexity, they become increasingly difficult to maintain.

2.5.3 Workflow orchestrators

Workflow orchestrators are tools that automate the scheduling, execution, and monitoring of multi-step data pipelines. Examples include Kedro [23], a Python framework that allows users to define their pipelines in code. It automatically infers execution order based on named inputs and provides a plugin for detailed visualization of pipeline execution information. However, it requires extensive configuration in YAML and learning the principles of a new framework.

Another workflow orchestrator is Apache Airflow [24], which defines workflows as Directed Acyclic Graphs (DAGs) in Python and includes a scheduler, retry logic, and a web UI. Running heavy computation inside Airflow itself is considered an anti-pattern, as it is intended to be used primarily for orchestration. This requires the user to offload heavy computation to

external systems and use Airflow only to trigger and monitor those tasks, which makes it not suitable for simple experimentation.

2.5.4 Task decorators

Python task decorators offer a lightweight, code-centric approach to building reproducible pipelines. A decorator transforms an ordinary Python function into a task that can be executed lazily, cached, and wired together with other tasks to form a computational graph. The decorator typically handles dependency resolution, memoization, and result persistence, allowing users to focus on writing pure functions while the decorator manages execution order and caching.

A notable example is joblib's `Memory.cache` decorator [25]. It automatically determines whether to execute a function based on the hashes of arguments and of the function's code and persists the results to disk. While useful for caching and persistence, joblib does not offer built-in output capture or a rich display of results like computational notebooks.

2.5.5 Summary

While many useful and popular tools for working with data exist and are discussed above, there does appear to be space for another one. A new tool called Caching Aware Executor (CEX) is proposed. The tool aims to provide a combination of functionality from the different categories discussed.

The first feature is the creation of an acyclic computational graph. When working with data and running experiments, it is common and often necessary to separate individual tasks such as loading, preprocessing, model training, result visualization, etc. into distinct steps or nodes. This provides the opportunity to reuse the results of one node in multiple subsequent nodes, for example, performing different statistical analyses and plotting the data obtained as a result of an experiment. For that purpose, the user should be allowed to define computational nodes and use them to create an arbitrary directed acyclic graph.

Second is cached/incremental execution of the computational graph. The development of experiments and data processing pipelines is inherently an iterative process. Because the execution of some nodes, such as loading data from storage, is time-consuming, it is necessary to cache the results in order to enable quick iteration. The tool must determine the proper order of execution and decide for each node based on factors such as previous input hashes and node code, whether it is necessary to execute the node again or to simply retrieve the cached result.

Last is output capture and rendering. One of the major benefits of computational notebooks is the immediate display of output produced by running cells. Captured output is composed of text in stdout and exceptions in stderr streams, as well as rich outputs produced by various libraries. including matplotlib plots, pandas dataframes, pillow images, and others. This is extremely convenient for both debugging and visualization of results. The tool will capture outputs per node, display them on demand, and allow for simple export so that outputs like plots can be preserved for future usage.

CHAPTER 3

Background

This chapter describes the necessary concepts used for estimating distance via RSSI. The effect of direction-dependent antenna gains is demonstrated, as well as the need for RSSI prefiltering. Also discussed are the models used for RSSI to distance conversion and the principal workings of the Kalman filter.

3.1 RSSI of WiFi signal

Received Signal Strength Indicator (RSSI) is typically a measure of the power of a received radio signal. In this thesis it is expressed as received signal power in decibel-milliwatts, a logarithmic unit referenced to the absolute power of 1 milliwatt, and the terms RSSI and received signal power are used interchangeably. The Institute of Electrical and Electronics Engineers (IEEE) 802.11 Standard [26], however, only defines it as a relative measure, the actual readings and their mapping to power in decibel-milliwatts may vary depending on the hardware vendor. The values in decibel-milliwatts also depend on transmitted power, so the values measured at the same distance will differ between, for example, WiFi and Bluetooth signals.

The general relationship between received signal power and distance in this thesis is modeled using Friis transmission equation in decibel form:

$$P_{\text{rx}} = P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} + 20\log_{10}\frac{\lambda}{4\pi d}, \tag{3.1}$$

where P_{rx} is the received power in decibel milli-watts, P_{tx} the transmitted power in decibel milli-watts, G_{rx} and G_{tx} the direction-dependent receiver and transmitter antenna gains in decibels referenced to an isotropic radiator, λ is the wavelength in meters, and d is the distance between antennas in meters.

Because received power is weaker than transmitted, RSSI values in dBm are typically negative. For the 2.4 GHz WiFi signal, used in this thesis and transmitted at 20 dBm power level, approximate idealized RSSI values, calculated using the Friis equation 3.1 at selected distances can be seen in Table 3.1 for reference. The values assume free space path loss and isotropic antenna gains. Realistic RSSI values are lower because they are affected by obstructions, antenna radiation patterns, and other factors.

Distance (m)	1	10	30	70	100
RSSI (dBm)	-20	-40	-50	-57	-60

Table 3.1: Idealized RSSI (dBm) values at selected distances (m)

When no information about the antenna gains or the environment path loss is given, the reference received power is used to improve accuracy and approximately calibrate the antenna gains. By separating the last term of the Friis equation 3.1:

$$P_{\text{rx}} = P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} + 20\log_{10}(\lambda) - 20\log_{10}(4\pi) - 20\log_{10}(d), \tag{3.2}$$

evaluating the expression at a reference distance d_0:

$$P_{\text{rx}}(d_0) = P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} + 20\log_{10}(\lambda) - 20\log_{10}(4\pi) - 20\log_{10}(d_0), \tag{3.3}$$

subtracting from Equation 3.2:

$$P_{\text{rx}}(d) - P_{\text{rx}}(d_0) = -20\log_{10}(d) + 20\log_{10}(d_0) = 20\log_{10}(\frac{d_0}{d}), \tag{3.4}$$

rearranging, introducing the path loss exponent and a zero-mean Gaussian random variable X_σ, in decibel-milliwatts, to represent shadowing:

$$P_{\text{rx}}(d) = P_{\text{rx}}(d_0) + 10 \cdot n \cdot \log_{10}(\frac{d_0}{d}) + X_\sigma, \tag{3.5}$$

we obtain the log-distance model [8] in terms of received power and distance. The RSSI dependent distance is then calculated as:

$$d = d_0 \cdot 10^{\frac{P_{\text{rx}}(d_0) - P_{\text{rx}}}{10 \cdot n}}. \tag{3.6}$$

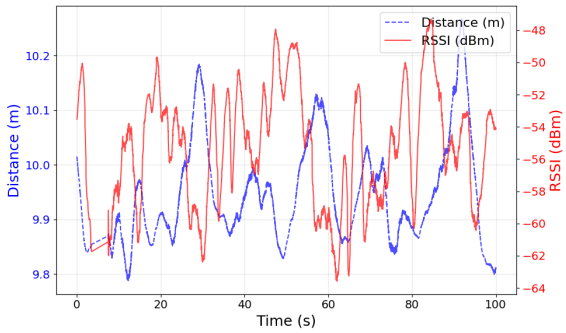
3.2 Joint radiation pattern

As described previously in Chapter 1, both antenna radiation patterns and possible obstruction of Line of Sight (LoS) can affect the direction-dependent antenna gains used to compute distance from RSSI. Together, the two gains form a joint radiation pattern, which depends on the mutual bearing of the Unmanned Aerial Vehicles (UAVs). Its effect can be demonstrated by examining one of the provided flight data recordings.

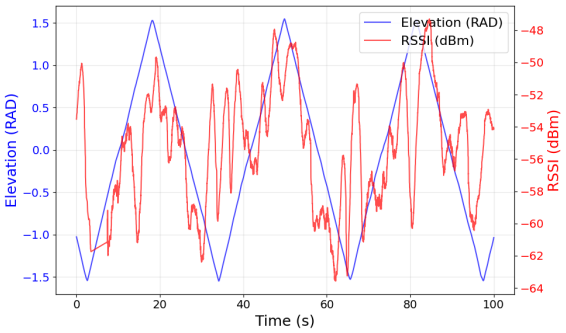
The data shows two UAVs flying in a vertical circle at high altitude over open space, maintaining a constant mutual distance of 10 meters. Additionally, each UAV rotates along its vertical axis within its body frame. Fig. 3.1a depicts the RSSI readings, smoothed by a Kalman filter, from the receiving UAV in decibel-milliwatts (right Y-scale) and distance to the transmitting UAV in meters (left Y-scale). It can be seen that while the distance remains nearly constant with only small fluctuations that are within decimeters, the smoothed RSSI values vary to the order of 10 decibel-milliwatts. The Fig. 3.1b shows the smoothed RSSI readings in decibel-milliwatts (right Y-scale) together with the relative elevation angle. While noisy, it can be seen that the RSSI values generally follow the elevation angle as the UAVs change their position on the vertical circle.

For the purposes of this thesis, the mutual bearing is derived from GNSS positioning and orientation data in the flight recordings. In a realistic deployment, they would have to be estimated based on the angle-of-arrival of the radio signal and communication with the transmitting UAV that would report its orientation with respect to a shared world frame or be provided by some other means.

It is proposed to model the joint radiation pattern based on real flight data. The model must take the mutual bearing, described by two pairs of azimuth-elevation angles and predict the



(a) Changing RSSI readings at constant distance between UAVs over time.



(b) RSSI readings trend following the relative elevation angle.

Figure 3.1: Effects of changes in relative bearing on the RSSI reading values

effective joint gain of the receiver and the transmitter. The predicted gain is then substituted in place of the two separate gains into the Friis equation:

$$P_{\text{rx}} = P_{\text{tx}} + G_{\text{joint}} + 20\log_{10}\frac{\lambda}{4\pi d}, \tag{3.7}$$

where the G_{joint} is the mutual bearing dependent joint radiation pattern in decibels referenced to an isotropic radiator, and the distance is then calculated as:

$$d = \frac{\lambda}{4\pi} \cdot 10^{\frac{P_{\text{tx}}+G_{\text{joint}}-P_{\text{rx}}}{10\cdot n}}. \tag{3.8}$$

Incorporating the joint radiation pattern prediction potentially compensates for the effects of direction dependent antenna gains and the obstruction of line-of-sight between the antennas by the body of either UAV that occurs in some mutual bearings.

3.3 Kalman filtering

In practice, due to multipath fading, obstruction, interference, and hardware factors, such as the changing temperature of electronic components, the received RSSI values fluctuate rapidly. This can be seen in Fig. 3.2, which shows raw RSSI readings from the receiving UAV in decibel-milliwatts (right Y-scale) and distance to the transmitting UAV in meters (left Y-scale), over time.

In worst cases, multiple fluctuations with the amplitude of about 10 dBms can be observed in the time span of less than a second. This makes raw RSSI readings unsuitable for direct conversion to distance, because the estimates would fluctuate by tens of meters. A method for smoothing the readings is required, and based on the experiment discussed in Section 2.3, the Kalman filter was selected, because it demonstrated the best performance in noise reduction.

The function of the Kalman filter can be described as a recursive algorithm that works in two phases: prediction and correction. The filter assumes that the true system evolves according to a linear stochastic difference model, and that measurements are linear functions of the state, both corrupted by zero-mean Gaussian noise.

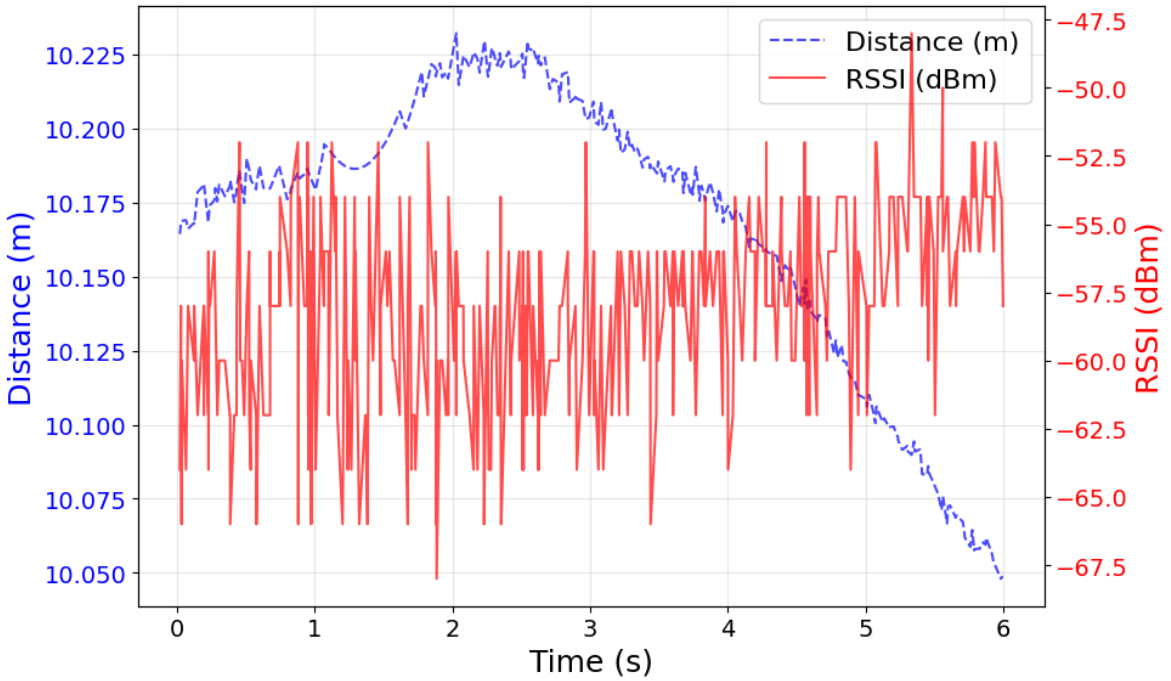


Figure 3.2: Fluctuating RSSI readings and ground truth distance between UAVs over time.

First we describe the system with n states as follows:

$$\begin{aligned} \mathbf{x}_k &= \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_{k-1} + \mathbf{w}_{k-1}, & \mathbf{w}_{k-1} &\sim \mathcal{N}(0, \mathbf{Q}) \\ \mathbf{z}_k &= \mathbf{H}\mathbf{x}_k + \mathbf{v}_k, & \mathbf{v}_k &\sim \mathcal{N}(0, \mathbf{R}) \\ \mathbf{P}_k &\in \mathbb{R}^{n \times n} \end{aligned} \tag{3.9}$$

where $\mathbf{x}_k \in \mathbb{R}^n$ is the system state vector at time k. These are the values we want to estimate. The available information is the measurement vector $\mathbf{z}_k \in \mathbb{R}^m$. The rest of the variables as defined as: $\mathbf{x}_{k-1} \in \mathbb{R}^n$ is the state at the previous time step, $\mathbf{A} \in \mathbb{R}^{n \times n}$ the state transition matrix, $\mathbf{B} \in \mathbb{R}^{n \times p}$ the control input matrix, $\mathbf{u}_{k-1} \in \mathbb{R}^p$ the known control vector, $\mathbf{w}_{k-1} \in \mathbb{R}^n$ the process noise (zero-mean Gaussian with covariance $\mathbf{Q} \in \mathbb{R}^{n \times n}$), $\mathbf{H} \in \mathbb{R}^{m \times n}$ the observation matrix, $\mathbf{P_k} \in \mathbb{R}^{n \times n}$ is the estimate uncertainty (covariance) matrix at time k, and $\mathbf{v}_k \in \mathbb{R}^m$ the measurement noise (zero-mean Gaussian with covariance $\mathbf{R} \in \mathbb{R}^{m \times m}$). The noise sequences $\mathbf{w}_{k-1}$ and $\mathbf{v}_k$ are mutually independent and also independent of the initial state.

Using the system model, the filter first predicts the next state by using the available current state to obtain an initial estimate.

$$\hat{\mathbf{x}}_k^- = \mathbf{A}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_{k-1} \tag{3.10}$$

where $\hat{\mathbf{x}}_k^- \in \mathbb{R}^n$ is the initial state estimate at time k, and $\hat{\mathbf{x}}_{k-1} \in \mathbb{R}^n$ is the previous corrected state estimate from the previous time step $k-1$. The estimate uncertainty matrix is also updated as follows.

$$\mathbf{P}_k^- = \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^\top + \mathbf{Q} \tag{3.11}$$

Here $\mathbf{P}_k^- \in \mathbb{R}^{n \times n}$ is the initial estimate uncertainty (covariance) matrix, $\mathbf{P}_{k-1} \in \mathbb{R}^{n \times n}$ is the corrected uncertainty matrix from time $k-1$, and $\mathbf{Q}$ is the process noise covariance defined earlier. We add $\mathbf{Q}$ because the estimate uncertainty grows naturally due to the process noise.

Then we correct the initial state and uncertainty estimates using the available information from the actual measurement $\mathbf{z}_k$. The first step is to compute the Kalman gain, which is an expression of the uncertainty in the prediction against the uncertainty in measurement.

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top \left(\mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R} \right)^{-1} \tag{3.12}$$

Next, the state estimate is corrected by adding the weighted difference between what was actually measured and what was predicted:

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \left(\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^- \right) \tag{3.13}$$

Here $\hat{\mathbf{x}}_k \in \mathbb{R}^n$ is the corrected state estimate at time k (the filtered result), $\mathbf{z}_k \in \mathbb{R}^m$ is the actual measurement, and the term $\left(\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^- \right)$ is called the innovation or residual.

Finally, the uncertainty of the corrected estimate is updated:

$$\mathbf{P}_k = \left(\mathbf{I} - \mathbf{K}_k \mathbf{H} \right) \mathbf{P}_k^- \tag{3.14}$$

where $\mathbf{P}_k \in \mathbb{R}^{n \times n}$ is the corrected estimate uncertainty (covariance) matrix, and $\mathbf{I} \in \mathbb{R}^{n \times n}$ is the identity matrix. The prediction and correction steps are then recursively repeated for all remaining measurements. This mechanism is then employed throughout this thesis to improve the distance estimate accuracy.

CHAPTER 4

Problem statement

This work aims to predict the distance between autonomous Unmanned Aerial Vehicles (UAVs) from RSSI measurements in terms of error-minimization statistical modeling.

The transmitted power samples $\mathbf{P}_{\text{tx}}$ from UAV a, the received power samples $\mathbf{P}_{\text{rx}}$ measured by UAV b, their mutual relative bearing samples, described by azimuth-elevation angles $\boldsymbol{\theta}^{\boldsymbol{tx}}_{\boldsymbol{rx}}, \boldsymbol{\phi}^{\boldsymbol{tx}}_{\boldsymbol{rx}}, \boldsymbol{\theta}^{\boldsymbol{rx}}_{\boldsymbol{tx}}, \boldsymbol{\phi}^{\boldsymbol{rx}}_{\boldsymbol{tx}}$ that express the direction of b with respect to (w.r.t) the body frame of a and the direction of a w.r.t the body frame of b respectively, their true relative distance $\boldsymbol{d}$, where bold symbols denote matrices, are available.

Formally, we aim to learn a model G_{joint}, that maps the four bearing angles to the joint radiation pattern:

$$G_{\text{joint}}(\theta^{tx}_{rx}, \phi^{tx}_{rx}, \theta^{rx}_{tx}, \phi^{rx}_{tx}) = G_{\text{tx}} + G_{\text{rx}}. \tag{4.1}$$

where G_{tx} is the direction-dependent transmitter antenna gain , the G_{rx} the direction dependent receiver antenna gain, and G_{joint} is any admissible function that takes four arguments and returns a scalar, so that the error:

$$E_{\text{rms}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\hat{d}_i - d_i\right)^2}, \tag{4.2}$$

is minimized. The $\hat{d}$ is the relative distance estimate obtained as:

$$\hat{d}_i = \frac{\lambda}{4\pi} \cdot 10^{\frac{P_{tx,i} + G(\theta^{tx}_{rx,i}, \phi^{tx}_{rx,i}, \theta^{rx}_{tx,i}, \phi^{rx}_{tx,i}) - P_{rx,i}}{10\, n}}, \tag{4.3}$$

where λ is the wavelength and n the path-loss exponent (e.g. $n = 2$ for free space).

The ultimate goal is to compensate for the direction-dependent attenuation that degrades RSSI-based ranging, thereby achieving a substantial improvement in distance accuracy compared to the isotropic-antenna baseline ($G = 0$).

CHAPTER 5

Proposed method

A multi-step processing pipeline is proposed to obtain a distance estimate. This section describes the data-processing steps and pipelines used to improve the distance estimate accuracy.

5.1 Pre-filtering RSSI

The raw RSSI stream is smoothed using the Kalman filter described in Section 3.3. The filter is configured as a one-dimensional random-walk model: the state x_k represents the true RSSI, evolving as $x_k = x_{k-1} + w_{k-1},\ w_{k-1} \sim \mathcal{N}(0,\, q\Delta t_k)$, and the raw measurement is $z_k = x_k + v_k$ with $v_k \sim \mathcal{N}(0, R)$. The process-noise variance is scaled by the elapsed time $\Delta t_k = t_k - t_{k-1}$ to account for the non-uniform sampling intervals of the flight logs.

The measurement covariance $\mathbf{R}$ was set to $5\,(\mathrm{dBm})^2$, the process covariance $\mathbf{Q}$ to $0.2\,(\mathrm{dBm})^2$ (reflecting the high ratio of noise in the measurement), the initial state x_0 was initialized as the first RSSI reading in the stream, and the initial uncertainty $\mathbf{P}$ was set to 10 dBm.

5.2 Joint gain predictor

The joint gain is predicted using a k-nearest neighbors (KNN) regressor, fitted using the provided flight data. Each recorded flight consists of two sets of logs, each for one of the two UAVs. A matrix of synchronized datapoints, derived from both of the sets, is created:

$$\mathbf{M}_{\text{data}} = \begin{bmatrix} \mathbf{t} & \mathbf{d} & \mathbf{P}_{\text{rx}} & \boldsymbol{\phi}_{\text{rx}}^{\text{tx}} & \boldsymbol{\theta}_{\text{rx}}^{\text{tx}} & \boldsymbol{\phi}_{\text{tx}}^{\text{rx}} & \boldsymbol{\theta}_{\text{tx}}^{\text{rx}} \end{bmatrix} \in \mathbb{R}^{N\times 7}, \tag{5.1}$$

where N is the number of datapoints for the flight, each bold symbol is a column vector of length N, and t is the time, d is the true distance in meters, P_{rx} is the RSSI at the receiver, $\phi_{\text{rx}}^{\text{tx}}$ and $\theta_{\text{rx}}^{\text{tx}}$ are the azimuth and elevation angles of the receiver as seen from the transmitter, $\phi_{\text{tx}}^{\text{rx}}$ and $\theta_{\text{tx}}^{\text{rx}}$ are the azimuth and elevation angles of the transmitter as seen from the receiver.

The input feature matrix is the angles $\begin{bmatrix} \boldsymbol{\phi}_{\text{rx}}^{\text{tx}} & \boldsymbol{\theta}_{\text{rx}}^{\text{tx}} & \boldsymbol{\phi}_{\text{tx}}^{\text{rx}} & \boldsymbol{\theta}_{\text{tx}}^{\text{rx}} \end{bmatrix} \in \mathbb{R}^{N\times 4}$ that describe the mutual bearing of the UAVs. The input features are scaled so that each column has a mean of 0 and a standard deviation of 1.

The target variable $\mathbf{G_{joint}} \in \mathbb{R}^{N\times 1}$ is the predicted joint gain, which is derived from the true distance $\mathbf{d} \in \mathbb{R}^{N\times 1}$ as:

$$G_{joint}^{i} = {P_{\text{rx}}}^{i} - P_{\text{tx}} - 20 \cdot \log_{10}\left(\frac{\lambda}{4\pi d^{i}}\right) \qquad \forall i \in \{1, \ldots, N\} \tag{5.2}$$

where P_{tx} is the known transmitted power (20 dBm).

The k parameter is tuned by creating a regressor for each value in the range of 10 to 200, with a step of 20, and comparing the average Root-Mean-Square Error (RMSE) of distance predictions.

5.3 Converting RSSI to distance

When converting RSSI to distance without predicting the joint gain, the log-distance model, shown in Equation 3.6, is used. The reference distance d_0 is set to one meter, n to 2, which corresponds to ideal line-of-sight propagation [8], and the reference power $P_{\text{rx}}(d_0)$ is fitted using ground truth distances d and the receiver RSSI recordings P_{rx}. The distance is then computed as follows:

$$d = 10^{\frac{P_{\text{fitted}} - P_{\text{rx}}}{10n}} \tag{5.3}$$

where P_{fitted} is the fitted reference power in decibel-milliwatts.

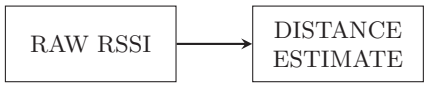
For pipelines featuring the joint gain predictor, the previously described Equation 3.2 is used.

5.4 Processing pipelines

Three RSSI processing pipelines for estimating distance were adopted. The first pipeline, simple raw RSSI to distance conversion, serves as a baseline for the Kalman filter performance. The second pipeline, pre-filtering RSSI followed by distance conversion, is used as a baseline to evaluate the performance of pipelines featuring the Joint Radiation Pattern (JRP) prediction and compensation. The last pipeline, where prefiltered RSSI is used to train a KNN regressor, is employed to improve the distance estimate accuracy by incorporating the Joint Radiation Pattern Compensation (JRPC).

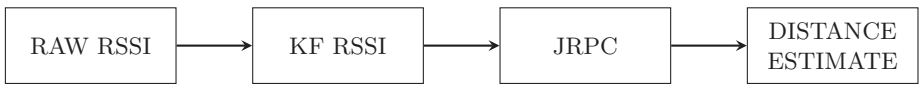
The adopted pipelines can be seen in Fig. 5.1, which displays:

1. Raw RSSI to distance conversion.
2. Pre-filtered RSSI to distance conversion.
3. Pre-filtered RSSI with JRPC to distance conversion.



(a) Estimation from raw RSSI

(b) Estimation from pre-filtered RSSI



(c) Estimation from pre-filtered RSSI with JRPC

Figure 5.1: Proposed processing pipelines for estimating distance via Received Signal Strength Indicator (RSSI)

CHAPTER 6

Implementation

This section provides an overview of the implementation of the custom tool Caching Aware Executor (CEX) proposed in Section 2.5 and the implementation of the RSSI processing pipelines proposed in Section 5.4 using CEX. The architecture and functionality of CEX is discussed, and concrete usage examples are given.

6.1 Caching Aware Executor

To provide the functionality discussed in Section 2.5, a custom tool was created. CEX is a Visual Studio Code (VSC) extension that allows the user to define a Directed Acyclic Graph (DAG) of computational nodes using a combination of Python code and drag-and-drop User Interface (UI). An example computational DAG with two nodes that load data, a data processor node, and a data plotter node can be seen in Fig. 6.1.

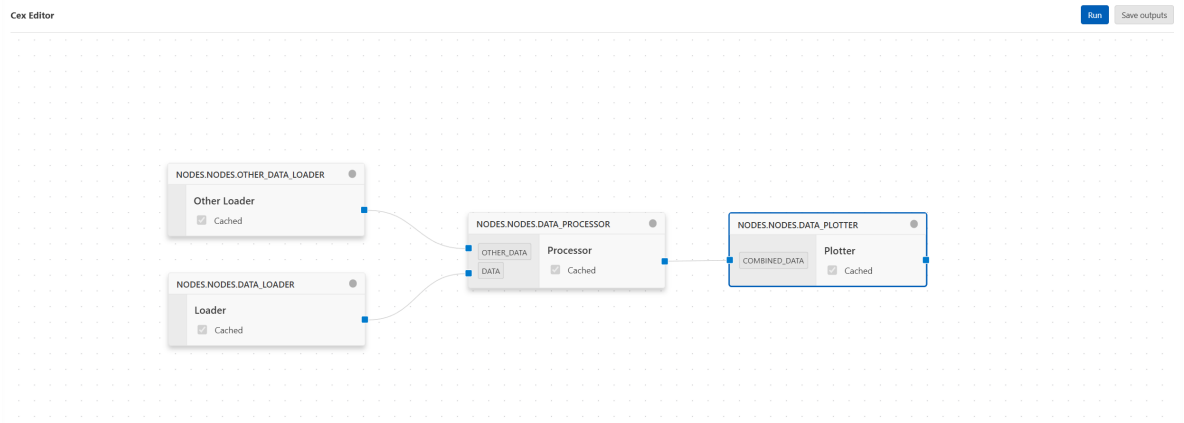


Figure 6.1: Example computational DAG in CEX

CEX provides cached incremental execution of the DAG, node output capture and output display. A CEX project is structured as shown in Fig. 6.2. The DAG is saved in a `CEX_WORKSPACE.json` file, saved outputs in the `artifacts` folder, and internal hashing data used for caching in the `.cex` folder and functions defining the CEX nodes in the `nodes` folder.

```
Project root
├── .cex ..................................................... Caching artifacts
├── artifacts ................................................ Outputs saved by CEX
├── nodes ..................Folder in which all files are scanned for the CEX decorators
└── CEX_WORKSPACE.json ....................................... Serialized DAG
```

Figure 6.2: Structure of a CEX project

To create a computational node, a function can be registered by marking it with a CEX decorator as shown in Listing 1.

```
from cex import cex

@cex.node(cache=True)
def data_loader():
    ...

@cex.node(cache=True)
def data_processor(data, other_data):
    ...

...
```

Listing 1: Registering a function with CEX node decorator

The marked functions are collected automatically and become available in the UI, where the user can use native VSC UI elements to create a node instance by entering a name and selecting its function. Any number of nodes can be instantiated from one collected function, reusing the same function code with potentially different data based on their position in the DAG.

Data is passed between nodes by examining the function signature, automatically creating an input port corresponding to each argument and an output for the value returned from the node. A close up in Fig. 6.3 shows a node instance created from the `Data_Processor` function defined in Listing 1. The figure shows that an input port is present for each of the `data` and `other_data` function arguments. A node UI element also displays the name of the node and the function it was created from.

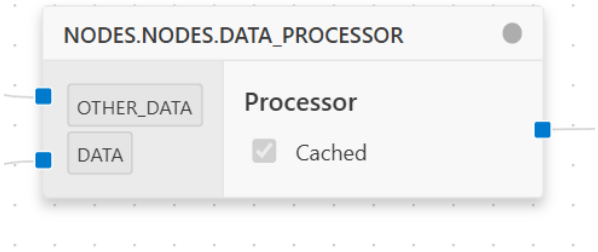


Figure 6.3: DAG node with multiple input ports

The edges of the DAG are then created by connecting the output ports to the input ports. The graph can be executed as a whole by pressing the run button. Caching can be enabled for each node, in which case the hashes of the node's inputs and code from the current run are compared to the stored hashes from the previous run. In case of a hash mismatch, the node is executed, otherwise an output from the previous run is reused. The hashes and individual node outputs are persisted between sessions by serializing and storing them, so that execution of nodes is skipped even in a new session unless it is necessary.

During the execution of each node, its outputs, such as plots, are captured. They are displayed in a separate panel, which can be opened by clicking the node. An example of a plot capture and rendering can be seen in Fig. 6.4. It is also possible to save the rich outputs for later use.

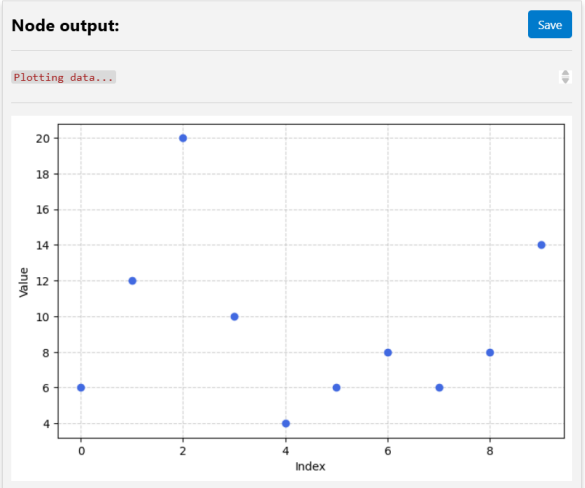


Figure 6.4: Example of rendering captured plot and text from stdout

6.2 Architecture overview

The CEX tool is composed of several software components that work together to deliver the features discussed in Section 6.1. As illustrated in the architecture diagram (Fig. 6.5), the main components are: a UI bundle built with React [27], a VSC extension host [28] and a core Python service bundled inside the VSC extension. The three-tier architecture was selected to potentially allow development and distribution via platforms other than VSC extensions. The core service is independent of any VSC specific Application Programming Interfaces (APIs) and can be potentially used with any other frontend.

The UI enables users to interact with the DAG - creating nodes and edges, initiating execution, and viewing or saving outputs. It sends commands related to DAG management and execution to the VSC extension via the provided VSC API [28]. The extension acts as a bridge between the UI and the Python service: it manages the service's lifecycle, passes the UI commands to the core service and serves the UI bundle inside a VSC tab. The core Python service handles function collection, DAG persistence, node execution order, caching, and output capture. The service exposes Remote Procedure Calls (RPCs) [29] that correspond directly to the UI commands for interacting with the DAG, its execution, and node outputs.

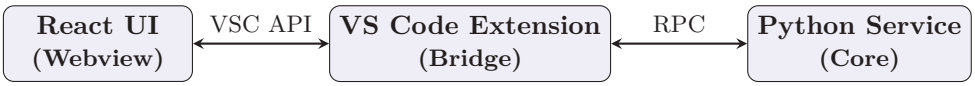


Figure 6.5: Architecture diagram of the CEX tool showing its main components and their communication methods.

A separate CEX Python package is provided so that the user functions can be marked with the CEX node decorators as seen in Listing 1. This package is imported in both the user's code and the core CEX service and defines a function registry singleton at the module top level. The collection itself is done by dynamically importing user modules into the core CEX service's Python runtime. When imported, decorators are automatically executed by the Python interpreter, which allows them to register their function reference and metadata inside the singleton function registry. The modules are re-imported periodically to keep the user's code up-to-date inside the CEX service.

However, the user's code itself may also import arbitrary Python modules, and the import will fail unless the Python paths (directories where Python searches for imported modules) are added to the corresponding environment variable of the CEX Python service. Therefore, it is necessary to perform a discovery of the user's Python environment. The discovery is initiated by the VSC extension, and happens every time the extension is activated (normally when opening a new VSC window). The basic procedure is as follows:

1. Inside the VSC extension, obtain the path to the user's Python interpreter from the VSC API.
2. Pass the interpreter path to the CEX service using RPC.
3. Inside the CEX service, create a separate process that uses the interpreter path to run a short script. This script lists the Python paths used by the user's interpreter, and the service collects those paths.
4. Add the collected paths to the CEX service environment variables.

The extension also listens for Python interpreter path changes and will repeat this process if a change of interpreter is detected. This allows CEX to dynamically import any modules referenced by the user's code.

The execution of the user's node instances is handled by first performing a topological sort, which guarantees that all of its ancestors have been executed prior to executing any given node itself. Caching and invalidation are then handled by the joblib's [25] `Memory.cache` decorator, applied by CEX to the user function. Before the function is executed, it is wrapped inside the IPython [20] `capture_output` context manager, which automatically captures the rich outputs. Both the function's own result and its rich outputs are then returned, causing the joblib to store them and retrieve them on cache hits.

The persistence of the DAG is achieved by serializing it and storing it in the project folder. The stored DAG is automatically updated any time it's changed by the user. Automatic export of all captured outputs is also provided by inferring the proper serialization mechanisms from the rich output mime type and storing it in the artifacts folder inside the project.

6.2.1 Limitations

While the adopted architecture is effective at providing the desired functionality for interactive, rapid experimentation and iteration when working with computational DAGs, it has several limitations. First, because the user's code is executed inside the CEX Python runtime, the user must use a compatible Python version. In case of version mismatch, unforeseen issues may arise when the user's code is executed. Second, because the only way CEX can identify a function is by its name, it is not possible to rename a function once a node using that function is included in the DAG. Third, to avoid searching and dynamically importing modules not defined by the users, such as third-party-packages, only files following the naming convention `node_*.py`, `*_node.py` or placed inside the `nodes` folder are scanned. Fourth, while an invalidation mechanism is provided by joblib, and will rerun a node if its inputs or code change, it cannot detect changes in functions called by the node. Last, the core CEX service is bundled as an executable. This means that multiplatform support requires compiling for each platform separately, and is not currently provided. Currently, CEX can only run on the Windows operating system.

6.3 RSSI Processing Pipelines

All steps necessary for the implementation of the pipelines proposed in Section 5.4 were implemented using the developed CEX tool. The total computational graph can be seen in Fig. 6.6.

The pipelines contain common preliminary nodes, executed in sequence.

- `Load flight data`: This node loads time, GNSS coordinates, RSSI readings, altitude, and orientation quaternions. It either extracts them from rosbags (if present) or loads the relevant data extract mentioned in Section 1.1.
- `Synchronize flight data`: Synchronizes the extracted data by finding the common time interval and performing one-dimensional linear interpolation.
- `Create flight features`: Transforms the absolute GNSS coordinates into mutual bearings.
- `Filter multipath datapoints`: Eliminates datapoints that are too close to the ground.

After this, the pipeline diverges into the three pipelines discussed in 5.4. Nodes that automate the plotting of figures and creation of tables are added alongside the Received Signal Strength Indicator (RSSI) processing.

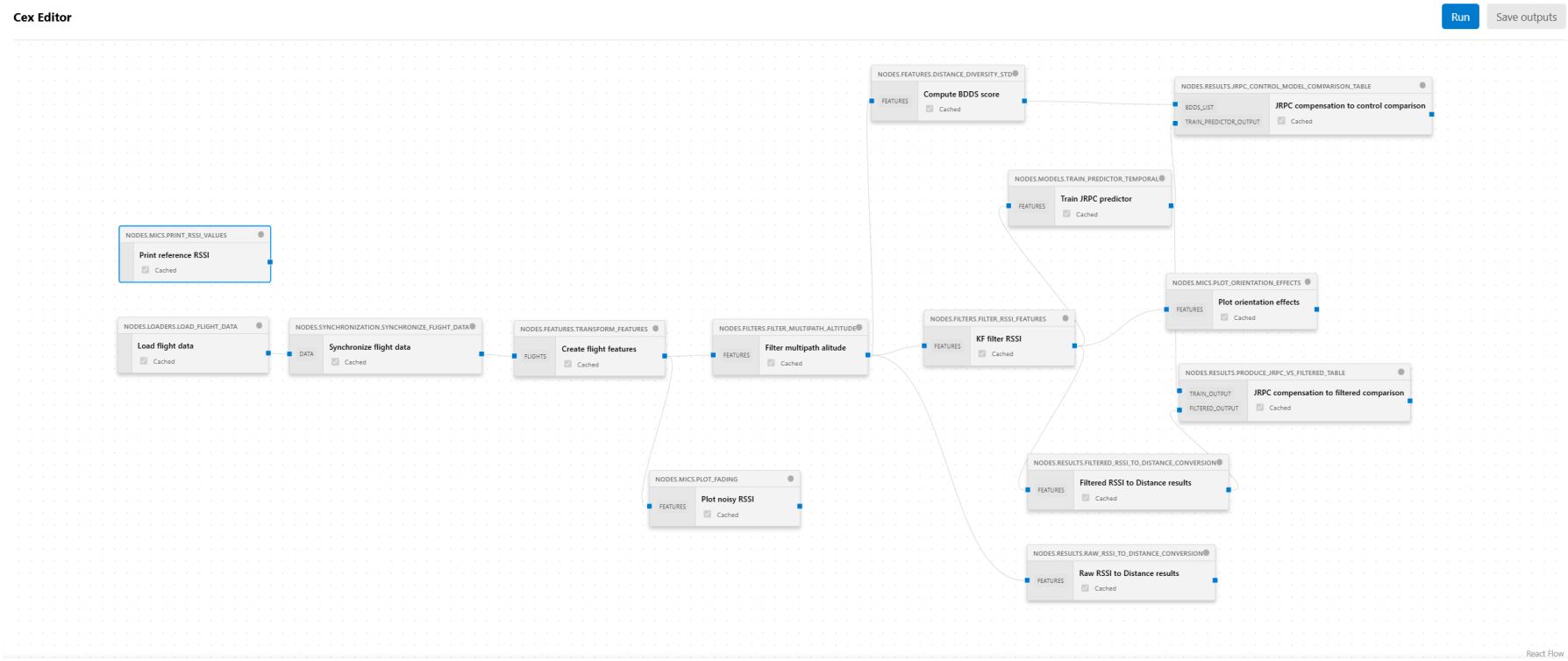


Figure 6.6: Total computational graph used for RSSI processing in CEX

CHAPTER 7

Results

The training and evaluation of the proposed processing pipelines described in Section 5.4 was performed using flight data collected for the study of antenna radiation patterns of a pair of quadrotor Unmanned Aerial Vehicles (UAVs) [30] and provided by the Multi-Robot Systems (MRS) group. The flights were conducted on a modular autonomous UAV platform [31] using a control and estimation system designed to support replicable research [32].

Multiple flights were performed and recorded, each flight consisting of a pair of UAVs. They can be categorized as follows:

- Calibration flights, where the two UAVs fly in a pre-set circular pattern, an example of which can be seen in Fig. 7.1, denoted as calibration flights (CF),
- Orthogonal flights, flown on a pre-set trajectory along orthogonal axis and denoted as (OF),
- Free flights, where the UAVs fly on an improvised random trajectory controlled manually, denoted as free flights (FF).

For each UAV in a given flight, a separate recording was provided in the form of a rosbag. Rosbag is a file format for storing serialized flight logs published by various elements such as sensors and flight control software. The recordings contain, among other data, raw received RSSI readings, orientation data, and GPS positioning data.

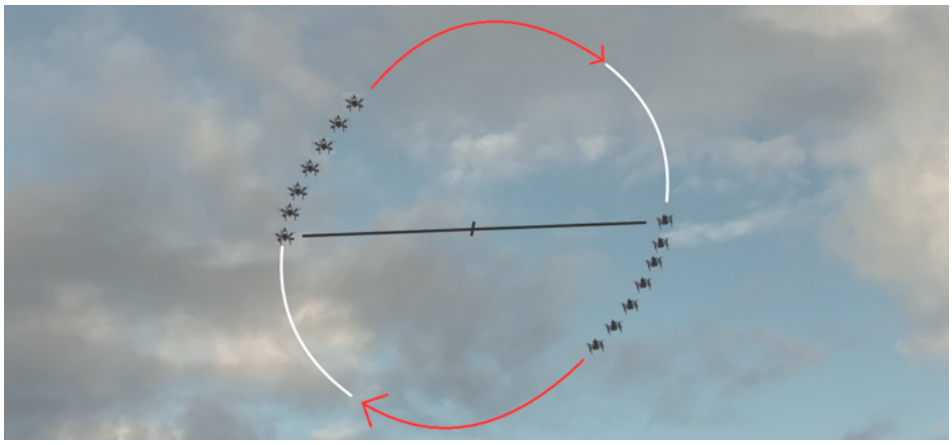


Figure 7.1: UAVs flying in a preset vertical circle calibration pattern at a constant mutual distance. Courtesy of the MRS group.

Provided flight recordings take place above open field terrain, with no obstruction present apart from a single tree over 50 meters away [30]. Not all flights were selected, as some were aborted, suffered crashes, or the recording files were corrupted. A breakdown of selected flights by terrain and piloting type, including a short description, can be seen in Table 7.1.

Name	Date & Time	Terrain	Piloting	Description
CF1	2025-10-07 17:19	Open field	Preset pattern	Flight at constant mutual distance in a vertical circle
CF2	2025-10-07 17:29	Open field	Preset pattern	Flight at constant mutual distance in a vertical circle
CF3	2025-10-07 18:07	Open field	Preset pattern	Flight at constant mutual distance in a vertical circle
CF4	2025-10-07 17:52	Open field	Preset pattern	Flight at constant mutual distance in a vertical circle
OF1	2025-10-08 10:48	Open field	Preset pattern	Flight at varying distance on orthogonal axis
OF2	2025-10-08 10:57	Open field	Preset pattern	Flight at varying distance on orthogonal horizontal axis
OF3	2025-10-08 13:38	Open field	Preset pattern	Flight at varying distance on a set horizontal axis
OF4	2025-10-08 14:09	Open field	Preset pattern	Flight at varying distance on a set horizontal axis
FF1	2025-10-09 13:41	Open field	Manual piloting	Flight at random distance and altitude around a stationary UAV
FF2	2025-10-09 14:20	Open field	Manual piloting	Flight at random distance and altitude around a stationary UAV
FF3	2025-10-09 17:25	Open field, buildings	Manual piloting	Flight in random pattern above buildings and open field

Table 7.1: Table of selected flights for the purpose of estimating distance via RSSI

The flights were done in several campaigns and varied significantly by the hardware used. The individual flights utilized different altitude control methods, with some flights being flown by barometer, which may have introduced error in the reported altitude. The means used to measure RSSI differed, with some flights using TP Link Archer T2U Plus WiFi USB module and others the internal WiFi module of the onboard computer Intel NUC10i7FNH/K as shown in Fig. 7.2a. The antenna orientation also varied with respect to (w.r.t) UAV frame [30], with either horizontal seen in Fig. 7.2c or vertical alignment seen in Fig. 7.2b. Because of the difference in used hardware, especially differences in antenna orientation, the model used for joint radiation pattern prediction cannot be expected to generalize across flights.

(a) Autonomous UAV with internal NUC WiFi connectivity. Courtesy of MRS group.

(b) Vertical external antenna orientation on autonomous UAV. Courtesy of MRS group.

(c) Horizontal external antenna orientation on autonomous UAV. Courtesy of MRS group.

Figure 7.2: Varied hardware used by UAVs.

7.1 Performance of distance estimates based on direct RSSI conversion

The performance of all distance estimates is evaluated using the average Root-Mean-Square Error (RMSE) which penalizes large outliers, Mean Absolute Error (MAE) that treats all deviations linearly, giving the typical error magnitude and the 95th Percentile Error (P95E) error, the highest error excluding the 5% of the largest errors (outliers) that provides an indication of worst-case behavior. The standard deviation of distance per flight is also considered.

The direct raw RSSI to distance conversion performance can be seen in Table 7.2, which displays the three error types for each individual flight as well as averaged over all flights. The best performance is shown on flight CF 3 and CF 4, with the respective RMSE errors of 8.67 m and 7.88 m. Flights OF 1 and OF 4 show much larger errors (RMSE of 43.22 m and 38.47 m). The average RMSE error is 22.90 m, MAE 13.56 m and P95E 43.19 m. The large difference between RMSE and MAE indicates the presence of severe outliers in the RSSI data. In general, the results show that direct conversion of raw RSSI to distance is not useful, which is expected as it is only meant to be a baseline for the pre-filtered RSSI to distance conversion performance.

Table 7.2: Raw RSSI to distance conversion errors per flight, baseline for the Kalman filter performance

Flight	RAW RMSE (m)	RAW MAE (m)	RAW P95E (m)
CF1	30.22	19.04	65.89
CF2	27.64	16.35	56.61
CF3	8.67	5.70	17.68
CF4	7.88	5.03	14.55
OF1	43.22	19.92	89.42
OF2	15.83	7.98	27.39
OF3	27.35	17.58	44.10
OF4	38.47	25.44	66.04
FF1	19.45	11.40	35.70
FF2	13.26	8.87	22.48
FF3	19.92	11.81	35.23
Average	**22.90**	**13.56**	**43.19**

The performance of pre-filtered RSSI to distance conversion can be seen in Table 7.3. Pre-filtering improves the overall metrics, bringing the average RMSE, MAE and P95E to 14.27 m, 9.98 m and 31.21 m respectively. Pre-filtering has suppressed the outliers, reducing the average RMSE by approximately 37.7% (from 22.90 m to 14.27 m) and bringing it much closer to the MAE, shrinking the gap from 9.34 m (raw) to 4.29 m (filtered). The MAE shows a modest reduction of 3.58 m (from 13.56 m to 9.98 m), and the P95E is reduced by 11.98 m (from 43.19 m to 31.21 m). All individual flights exhibit lower errors after filtering. In general, the persistently high overall error metrics indicate that the effects of the joint radiation pattern dominate over random noise and confirm the need for a more sophisticated approach.

Table 7.3: Filtered RSSI to distance conversion errors per flight, baseline for pipelines with joint radiation pattern compensation

Flight	Filtered RMSE (m)	Filtered MAE (m)	Filtered P95E (m)
CF1	18.09	13.24	39.81
CF2	16.32	11.48	38.58
CF3	4.38	3.52	7.32
CF4	4.10	3.53	6.57
OF1	29.53	16.39	73.52
OF2	6.93	4.91	14.31
OF3	18.05	14.00	34.93
OF4	26.31	19.02	59.56
FF1	11.24	7.89	24.25
FF2	8.37	6.64	14.09
FF3	13.67	9.19	30.41
Average	**14.27**	**9.98**	**31.21**

7.2 Performance of distance estimates with joint radiation pattern compensation

The high overall errors of pre-filtered RSSI to distance conversion indicate that the joint radiation pattern effects are a major source of error. It is supposed that by predicting the joint radiation pattern and incorporating it into the distance calculation, the overall error metrics can be significantly reduced.

The performance of distance estimation with Joint Radiation Pattern Compensation (JRPC) is evaluated in two ways. First, by directly comparing the overall RMSE, MAE, and P95E to the pre-filtered RSSI to distance conversion baseline.

Second, in some flights, knowledge of the mutual bearing alone is sufficient to accurately predict the distance. This occurs because the variation in distance for similar mutual bearings is small. For example, in the calibration flights, the distance is nearly constant, causing the distance for all mutual bearings to deviate by only a few decimetres. To ascertain that the JRPC model learns a distance-invariant pattern (as opposed to a non-causal, deterministic mapping between bearing and gain that only works for a specific fixed distance), a control model that directly predicts distance from mutual bearing, disregarding RSSI is employed. It is expected that the control model will improve accuracy on flights where the relationship between mutual bearing and distance is nearly deterministic, and will significantly degrade it where it is not, while the JRPC model will improve accuracy across all flights.

To quantify how much the mutual bearings determine the distance in a given flight, a bearing-distance diversity score Bearing Distance Diversity Score (BDDS) was devised. It is computed as follows:

1. From the flight's data points, extract the four bearing angles:

$$\phi_{\mathrm{rx}}^{\mathrm{tx}},\ \theta_{\mathrm{rx}}^{\mathrm{tx}},\ \phi_{\mathrm{tx}}^{\mathrm{rx}},\ \theta_{\mathrm{tx}}^{\mathrm{rx}}.$$

2. Build an auxiliary k-nearest neighbors model ($k = 100$) in this feature space using Euclidean distance.

3. For each point, find its 100 nearest neighbors (including itself). Take the ground-truth GPS distances of the 99 other neighbors and compute their standard deviation. This yields a local measure of how much the true distance varies in the flight data for a given bearing.

4. Average these local standard deviations across all points to get a single "bearing-distance diversity score" (BDDS) for the flight.

The per-flight errors for both the JRPC model and the control model are obtained using a leave-segment-out cross-validation scheme:

1. Split the flight's time-series data into 10 temporally contiguous segments of equal length,

2. Sequentially treat each segment as the test set, while the remaining 9 segments serve as the training set,

3. For each fold, train the model on the 9 training segments and evaluate it on the held-out test segment

4. Compute the RMSE, MAE and P95E on the test segment,

the average of the fold-wise errors is then reported as the final error for each flight. Because the flight data are densely sampled over time, this temporal segmentation provides a more realistic validation than a random train/test split. The same 10-fold procedure is used for both models, ensuring a fair comparison.

The performance of the JRPC model and the control model can be seen in Table 7.4. The results show that while the control model achieves very low error on flights with low BDDS (e.g., CF 3 with 0.16 m BDDS gives control RMSE of 0.74 m), its error increases dramatically when the BDDS is high (OF 4: 15.83 m BDDS gives control RMSE of 37.81 m). In contrast, the JRPC model maintains consistently moderate errors across all flights, with an average RMSE of 7.60 m compared to 9.94 m for the control model. On flights with high bearing-distance diversity (OF 3, OF 4, FF 1, FF 3), the JRPC model reduces the control model's RMSE by factors of 1.4 to 3.0, showing that it successfully learns a distance-invariant gain pattern rather than a simple deterministic mapping from bearing to distance.

Table 7.4: JRPC Model and control model RMSE per flight with BDDS

Flight	JRPC RMSE (m)	Control RMSE (m)	BDDS (m)
CF1	4.72	0.89	0.27
CF2	6.56	1.40	0.31
CF3	5.73	0.74	0.16
CF4	5.21	1.73	0.43
OF1	7.95	5.75	1.71
OF2	5.07	5.90	1.01
OF3	14.46	20.51	4.76
OF4	12.65	37.81	15.83
FF1	7.42	14.20	7.21
FF2	5.01	5.83	2.80
FF3	8.88	14.56	8.20
Average	**7.60**	**9.94**	**-**

The comparison between the JRPC model and the filtered RSSI baseline (Table 7.5) shows a substantial improvement across all error metrics. On average, the JRPC model reduces the RMSE from 14.27 m to 7.60 m (a 46.7% reduction), the MAE from 9.98 m to 4.70 m (a 52.9% reduction), and the P95E from 31.21 m to 15.23 m (a 51.2% reduction). On flights where the baseline errors were largest, the reductions are especially pronounced: for OF 4, RMSE drops from 26.31 m to 12.65 m, MAE from 19.02 m to 6.41 m, and P95E from 59.56 m to 23.34 m; for FF 1, RMSE drops from 11.24 m to 7.42 m, MAE from 7.89 m to 5.18 m, and P95E from 24.25 m to 16.20 m. On flight CF 3, however, the JRPC model performed slightly worse than the baseline, with RMSE increasing from 4.38 m to 5.73 m, MAE from 3.52 m to 3.80 m, and P95E from 7.32 m to 12.65 m. While substantial ranging errors are still present, these results indicate that compensating for the joint radiation pattern drastically reduces the dominant error source.

Table 7.5: Comparison of filtered RSSI to distance with JRPC model and direct filtered RSSI to distance conversion baseline errors per flight

Flight	Filtered RSSI to distance with JRPC			Direct filtered RSSI to distance baseline		
	RMSE (m)	**MAE (m)**	**P95E (m)**	**RMSE (m)**	**MAE (m)**	**P95E (m)**
CF1	4.72	3.56	8.82	18.09	13.24	39.81
CF2	6.56	4.23	12.87	16.32	11.48	38.58
CF3	5.73	3.80	12.65	4.38	3.52	7.32
CF4	5.21	3.45	11.20	4.10	3.53	6.57
OF1	7.95	4.40	14.01	29.53	16.39	73.52
OF2	5.07	2.72	10.15	6.93	4.91	14.31
OF3	14.46	7.99	30.43	18.05	14.00	34.93
OF4	12.65	6.41	23.34	26.31	19.02	59.56
FF1	7.42	5.18	16.20	11.24	7.89	24.25
FF2	5.01	4.03	10.08	8.37	6.64	14.09
FF3	8.88	5.91	17.82	13.67	9.19	30.41
Average	**7.60**	**4.70**	**15.23**	**14.27**	**9.98**	**31.21**

CHAPTER 8

Conclusion

This thesis investigated the feasibility of improving RSSI-based distance estimation between autonomous UAVs by predicting the combined radiation pattern from the relative bearing of the two aircraft. A multi-step processing pipeline was proposed, consisting of Kalman-filter-based RSSI pre-filtering and a k-nearest neighbors regressor that maps the four mutual bearing angles to the joint radiation pattern correction term.

The pipeline was evaluated on real flight recordings provided by the Multi-Robot Systems group. Within individual flights, the results appear promising. Pre-filtering the raw RSSI stream with a Kalman filter reduced the average RMSE from approximately 22.90 m to 14.27 m, a reduction of approximately 38%. Incorporating the gain prediction further reduced the average RMSE to approximately 7.60 m, which represents approximately a factor-of-two improvement over the pre-filtered baseline. Furthermore, the comparison with a control model that directly predicts distance from the bearing angles suggests that the gain predictor is learning a meaningful mapping from relative orientation to antenna gain rather than merely exploiting a deterministic relationship between bearing and distance.

However, an important limitation must be acknowledged. The evaluation was performed exclusively using a leave-segment-out cross-validation scheme within each individual flight. Cross-flight validation, where a model trained on one flight would be tested on a different flight, was not feasible due to significant hardware differences between the recorded flights. As a consequence, the reported improvements should be interpreted with caution. Additionally, while a substantial error reduction was achieved, the absolute error magnitude still remains too high for most practical uses requiring precision.

In terms of the secondary contribution of this work, existing tools for reproducible research were reviewed, and a new computational graph creation tool was proposed and used to develop all data-processing steps. Reproducibility of the joint radiation pattern prediction model creation and evaluation was prioritized, ensuring that the reported results can be independently reproduced from the provided code and data.

For future work, it would be valuable to conduct a dedicated flight campaign with a consistent hardware configuration across multiple flights. This would enable proper cross-flight validation and provide a more reliable assessment of the effectiveness of using a joint radiation pattern prediction model for error reduction of an Received Signal Strength Indicator (RSSI) based distance estimate.

References

[1] J. Gärtner. “UAV formation keeping based on mutual distance measurement”. MA thesis. Prague, Czech Republic: Czech Technical University in Prague, June 2021.

[2] R. Yokoyama, B. Kimura, and E. Moreira. “An architecture for secure positioning in a UAV swarm using RSSI-based distance estimation”. In: *ACM SIGAPP Applied Computing Review* 14 (June 2014), pp. 36–44.

[3] J. A. Gutierrez, E. H. Callaway, and R. L. Barrett. “IEEE Standard 802.15.4”. In: *Low-Rate Wireless Personal Area Networks: Enabling Wireless Sensors with IEEE 802.15.4*. 2007.

[4] A. Marut, K. Wojtowicz, and K. Falkowski. “ArUco markers pose estimation in UAV landing aid system”. In: *2019 IEEE 5th International Workshop on Metrology for AeroSpace (MetroAeroSpace)*. 2019, pp. 261–266.

[5] S. Li and H. Gao. “Propagation characteristics of 2.4GHz wireless channel in cornfields”. In: *2011 IEEE 13th International Conference on Communication Technology*. 2011, pp. 136–140.

[6] O. Elijah et al. “Effect of Weather Condition on LoRa IoT Communication Technology in a Tropical Region: Malaysia”. In: *IEEE Access* 9 (2021), pp. 72835–72843.

[7] T. Soejima. “Fresnel gain of aperture aerials”. In: *Proceedings of the Institution of Electrical Engineers* 110 (6 1963), pp. 1021–1027.

[8] T. Rappaport. *Wireless Communications: Principles and Practice*. Prentice Hall communications engineering and emerging technologies series. Cambridge University Press, 2024. ISBN: 9781009489836.

[9] The Turing Way Community. *The Turing Way: A handbook for reproducible, ethical and collaborative research*. https://book.the-turing-way.org. 2025.

[10] C. L. Sang et al. “An Analytical Study of Time of Flight Error Estimation in Two-Way Ranging Methods”. In: *2018 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*. 2018, pp. 1–8.

[11] J. Supramongkonset et al. “Empirical path loss channel characterization based on air-to-air ground reflection channel modeling for UAV-enabled wireless communications”. In: *Wirel. Commun. Mob. Comput.* 2021.1 (Jan. 2021), pp. 1–10.

[12] S. Duangsuwan and P. Jamjareegulgarn. “Exploring ground reflection effects on received signal strength indicator and path loss in far-field air-to-air for unmanned aerial vehicle-enabled wireless communication”. In: *Drones* 8.11 (Nov. 2024), p. 677.

[13] E. Pagliari, L. Davoli, and G. Ferrari. “Wi-Fi-Based Real-Time UAV Localization: A Comparative Analysis Between RSSI-Based and FTM-Based Approaches”. In: *IEEE Transactions on Aerospace and Electronic Systems* 60.6 (2024), pp. 8757–8778.

[14] R. S. Rosli, M. H. Habaebi, and M. R. Islam. “Comparative Analysis of Digital Filters for Received Signal Strength Indicator”. In: *2018 7th International Conference on Computer and Communication Engineering (ICCCE)*. 2018, pp. 214–217.

[15] Bluetooth SIG, Inc. *Bluetooth Core Specification*. https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-62/out/en/index-en.html. Version 6.2. Accessed: 2026-05-14. 2025.

[16] D. Csik et al. "Impact of Antenna Orientation on Localization Accuracy Using RSSI-based Trilateration". In: *Analecta Technica Szegedinensia* 18 (Aug. 2024), pp. 22–29.

[17] D. Lundgren. *Method and system for antenna orientation compensation for power ranging*. U.S. Patent Application US20110163917A1. Filing date: December 30, 2009. July 2011.

[18] T. Kluyver et al. "Jupyter Notebooks - a publishing format for reproducible computational workflows". In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, 2016.

[19] A. Agrawal and M. Scolnick. *marimo - an open-source reactive notebook for Python*. https://github.com/marimo-team/marimo. Aug. 2023.

[20] F. Perez and B. E. Granger. "IPython: A System for Interactive Scientific Computing". In: *Computing in Science & Engineering* 9.3 (2007), pp. 21–29.

[21] J. F. Pimentel et al. "Understanding and improving the quality and reproducibility of Jupyter notebooks". In: *Empir. Softw. Eng.* 26.4 (May 2021), p. 65.

[22] R. Stallman, R. McGrath, and P. D. Smith. *GNU make: A program for directed recompilation: GNU make version 3.81*. GNU Press, 2004.

[23] S. Alam et al. *Kedro*. https://github.com/kedro-org/kedro. Version 1.3.1. Apr. 2026.

[24] Apache Software Foundation. *Apache Airflow*. https://airflow.apache.org. Accessed: 2026-05-21. 2025.

[25] The joblib developers. *joblib*. Version 1.3.1.

[26] "IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks–Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications". In: *IEEE Std 802.11-2024 (Revision of IEEE Std 802.11-2020)* (2025), pp. 1–5956.

[27] React Team. *React Reference Overview*. Official React documentation (reference overview). Meta Open Source. 2026. URL: https://react.dev/reference/react (visited on 05/21/2026).

[28] Microsoft. *Extension API*. Visual Studio Code documentation on the extension platform and APIs. Microsoft. 2026. URL: https://code.visualstudio.com/api (visited on 05/21/2026).

[29] C. Tiller et al. *GRPC/GRPC*. https://github.com/grpc/grpc. May 2026.

[30] M. Zoula et al. *Orientation Matters: Learning Radiation Patterns of Multi-Rotor UAVs In-Flight to Enhance Communication Availability Modeling*. 2026. arXiv: 2604.02827 [cs.RO].

[31] D. Hert et al. "MRS Drone: A Modular Platform for Real-World Deployment of Aerial Multi-Robot Systems". In: *Journal of Intelligent & Robotic Systems* 108.4 (2023). ISSN: 1573-0409.

[32] T. Baca et al. "The MRS UAV system: Pushing the frontiers of reproducible research, real-world deployment, and education with autonomous unmanned aerial vehicles". In: *J. Intell. Robot. Syst.* 102.1 (May 2021).

Appendix A
List of Attachments

The following files are attached to this thesis.

cex_tool.zip
: Source code of the Caching Aware Executor (CEX) tool.

rssi_project.zip
: Source code of the project used for the development of the RSSI processing pipelines proposed in this thesis. Includes the compiled Visual Studio Code (VSC) extension of the CEX tool, usage instruction and an installation script.